

Segment-6

Contents

1. Virtual Functions
2. Pointers to derived classes,
3. Applying Polymorphism using virtual functions,
4. Polymorphic class,
5. Pure Virtual functions,
6. Abstract classes,
7. Early binding, and late binding.

Pointers to derived classes:

- A **Pointer** declared as a **Pointer** to a base class can also be used to point to any class derived from that base. For example: assume two classes called base and derived, where derived inherits base.
- Base pointer can point to an object of any class which is derived from that base class. The reverse is not true. That is, a pointer of the derived class cannot be used to access an object of the base class.
- Although you can use a base pointer to point to a derived object, you can access only those members of the derived object that were inherited from the base. This is because the base pointer has knowledge only of the base class. It knows nothing about the members added by the derived class.

```
base *p; // base class pointer
base base_ob ; // object of type base
derived derived_ob ; // object of type derived
// p can point to base objects
p = &base_ob ; // p points to base object
// p can also point to derived objects without error
p = &derived_ob ; // p points to derived object
```

Here is a short program that illustrates how a base class pointer can be used to access a derived class:

```
#include <iostream>
using namespace std;
class base
{
    int x;
public :
    void setx (int i) {
        x = i;
    }
    int getx () {
        return x;
    }
}
```

```

    }
};
class derived : public base
{
int y;
public :
void sety (int i) {
y = i;
}
int gety () {
return y;
}
};
int main ()
{
base *p; // pointer to base type
base b_ob ; // object of base
derived d_ob ; // object of derived
// use p to access base object
p = & b_ob ;
p-> setx (10) ; // access base object. -> This syntax is commonly used in object-
oriented programming languages like C++ to access member functions and
variables of an object through a pointer to that object.
cout << " Base object x: " << p-> getx () << '\n';
// use p to access derived object
p = &d_ob ; // point to derived object
p-> setx (99) ; // access derived object
// can 't use p to set y, so do it directly
d_ob . sety (88) ;
cout << " Derived object x: " << p-> getx () << '\n';
cout << " Derived object y: " << d_ob . gety () << '\n';
return 0;
}

```

Introduction to Virtual Function:

- A virtual function is a member function that is declared within a base class and redefined by a derived class. To create a virtual function, precede the function's declaration with the keyword **virtual**. When a virtual function is redefined by a derived class, the keyword **virtual** is not needed.
- When a base pointer points to a derived object that contains a virtual function and that virtual function is called through that pointer, C++ determines which version of that function will be executed based upon the type of object being pointed to by the pointer. And, this determination is made at run time.

Example:

```
class Base //base class
{
public:
virtual void show() //virtual function
{
    cout << "Base\n"; }
};
class Derv1 : public Base //derived class 1
{
public:
void show()
{
    cout << "Derv1\n"; }
};
class Derv2 : public Base //derived class 2
{
public:
void show()
{
    cout << "Derv2\n"; }
};
int main()
{
    Derv1 dv1; //object of derived class 1
    Derv2 dv2; //object of derived class 2
    Base* ptr; //pointer to base class
    ptr = &dv1; //put address of dv1 in pointer
    ptr->show(); //execute show() of derive1
    ptr = &dv2; //put address of dv2 in pointer
    ptr->show(); //execute show() of derive 2
    return 0;
}
```

Here is a short example that uses a virtual function:

// A simple example using a virtual function .

```
# include <iostream >
```

```
using namespace std;
```

```
class base
```

```
{
```

```
public :
```

```
int i;
```

```
base (int x) { i = x; }
```

```
virtual void func ()
```

```
{
```

```
cout << " Using base version of func (): ";
```

```

cout << i << '\n';
}
};
class derived1 : public base
{
public :
derived1 (int x) : base (x) {}
void func ()
{
cout << " Using derived1 's version of func (): ";
cout << i*i << '\n';
}
};
class derived2
: public base
{
public :
derived2 (int x) : base (x) {}
void func ()
{
cout << " Using derived2 's version of func (): ";
cout << i+i << '\n';
}
};
int main ()
{
base *p;
base ob (10) ;
derived1 d_ob1 (10) ;
derived2 d_ob2 (10) ;
p = &ob;
p->func (); // use base 's func ()
p = &d_ob1 ;
p->func (); // use derived1 's func ()
p = &d_ob2 ;
p->func (); // use derived2 's func ()
return 0;
}

```

This program displays the following output:

Using base version of func(): 10

Using derived1's version of func(): 100

Using derived2's version of func(): 20

Another Example:

```
# include <iostream >
using namespace std;
```

```
class sound{
public:
    virtual void print(){
        cout<<"Sound"<<endl;
    }
};
```

```
class dog:public sound{
void print(){
    cout<<"Dog Dog"<<endl;
}
};
```

```
class cat:public sound{
void print(){
    cout<<"Cat Cat"<<endl;
}
};
```

```
int main(){
    sound s;
    dog d;
    cat c;
```

```
    sound *str;
    str=&d;
    str->print();
```

```
    str=&c;
    str->print();
    return 0;
}
```

Output:

Dog Dog
Cat Cat

Pure Virtual Functions and Abstract Classes in C++

A pure virtual function is a virtual function in C++ for which we need not to write any function definition and only we have to declare it. It is declared by assigning 0 in the declaration.

- An abstract class is a class in C++ which have at least one pure virtual function.
- Abstract class can have normal functions and variables along with a pure virtual function.
- Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
- If an Abstract Class has derived class, they must implement all pure virtual functions, or else they will become Abstract too.
- When a class contains at least one pure virtual function, it is referred to as an *abstract class*. Since an abstract class contains at least one function for which no body exists, it is, technically, an incomplete type, and no objects of that class can be created.

```
#include<iostream>
using namespace std;
class B {
    public:
        virtual void s() = 0; // Pure Virtual Function};
class D:public B {
    public:
        void s() {
            cout << "Virtual Function in Derived class\n";
        };
};
int main() {
    B *b;
    D dobj;
    b = &dobj;
    b->s();
}
```

Output

Virtual Function in Derived class

```

class Base //base class
{
public:
virtual void show() = 0; //pure virtual function
};
class Derv1 : public Base //derived class 1
{
public:
void show()
{ cout << "Derv1\n"; }
};
class Derv2 : public Base
{//derived class 2
public:
void show()
{
cout << "Derv2\n"; }
};

int main()
{
Base* arr[2]; //array of pointers to base class
Derv1 dv1; //object of derived class 1
Derv2 dv2; //object of derived class 2
arr[0] = &dv1; //put address of dv1 in array
arr[1] = &dv2; //put address of dv2 in array
arr[0]->show(); //execute show() in both objects
arr[1]->show();
return 0;
}

```

Output:

Derv1

Derv2

- Here the virtual function show() is declared as virtual void show() = 0; // pure virtual function
- The equal sign here has nothing to do with assignment; the value 0 is not assigned to anything.
- The =0 syntax is simply how we tell the compiler that a virtual function will be pure.
- Polymorphic Class
-

Polymorphic class:

A **class** having at least one virtual function is called a **polymorphic** class. In object-oriented programming, a polymorphic class is a class that can be used as a base class for other derived classes, and can be accessed through a pointer or reference to its base class type. Polymorphic classes are typically designed with

virtual functions, which allow the derived classes to override the behavior of the base class functions.

For example, consider a base class called **Shape**, which has a virtual function called **draw()** that can be overridden by its derived classes such as **Circle** and **Rectangle**. A polymorphic class can be instantiated as an object of any of these derived classes, and accessed through a pointer or reference to the base class type.

```
class Shape {
public:
    virtual void draw() = 0; // pure virtual function
};
```

```
class Circle : public Shape {
public:
    void draw() {
        // draw circle
    }
};
```

```
class Rectangle : public Shape {
public:
    void draw(){
        // draw rectangle
    }
};
```

```
int main() {
    Circle c;
    Rectangle r;
    Shape* ptr;
```

```
    ptr = &c; // assign pointer to Circle object
    ptr->draw(); // call draw() on Circle object
```

```
    ptr = &r; // assign pointer to Rectangle object
    ptr->draw(); // call draw() on Rectangle object
```

```
    return 0;
}
```


Binding :

When we compile our program, all variables and functions are mapped to addresses in memory. These addresses are then used for reference in the program. This process of converting variables and functions into addresses is known as **binding**.

There are two types of binding such as **Early Binding and Late Binding**

Early binding:

Early binding is also called **static binding**. Early binding occurs when we make the explicit or direct function call in our program.

When the compiler encounters a function call in the program, it replaces that function call with a machine language instruction to go to that function.

Therefore, when the point of execution comes in that function call line, it reads that instruction and then jumps to the function given by memory address.

Early bound entities include normal functions, overloaded functions, and non-virtual member and friend functions. When these types of functions are compiled, all address information necessary to call them is known at compile time.

Consider the following code:

```
#include<iostream>
using namespace std;
void display(int x){
    cout << x;
}
int main(){
    display(5);
    return 0;
}
```

Late binding:

Late binding is also called **dynamic binding**. Late binding occurs when we make implicit or indirect function calls in our program. An example of this is using function pointers or virtual functions when using classes. Here, the function call's memory reference is not determined at compile-time, but rather at run-time. In C++, a virtual function is a late bound object.

Consider the following code:

```
#include<iostream>
using namespace std;
void display(int x){
```

```

cout << x;
}
int main() {
    void (*ptrFunc)(int) = display; // define a function pointer type
    ptrFunc(5);
    return 0;
}

```

In this example, the compiler does not know which function is stored in that function pointer until the runtime, and therefore cannot do the early binding.

- If the compiler knows at the compile-time which function is called, it is called early binding.
- If a compiler does not know at compile-time which functions to call up until the run-time, it is called late binding

Previous Question Practice

- 3 a) Write a complete program that shows the uses of virtual function. 4
- b) What do you know about early binding and late binding? Discuss the pros and cons of them. 3

- c) Correct the errors in the following program

```

class test
{
    private:
        int m;
    public:
        void getdata()
        {
            cout<<"Enter number";
            cin>> m;
        }
        void display()
        {
            cout << m;
        }
};

main()
{
    test T;
    T->getdata();
    T->display();
    Test *p;
    P=new test;
    p.getdata();
    (*p).display();
}

```

- 3.a) What is *virtual function*? Why do we need virtual functions? 2
- b) What are the reasons to use *pure virtual functions*? Write a program by using pure virtual function. 4
- c) What do you mean by *polymorphism* in OOP? Write the difference between early binding and late binding. Give examples of static and dynamic polymorphism. 4
-
- 3.a) What is virtual function? Explain with example. 3
- b) What do you know about early binding and late binding? Discuss the pros and cons of them. 3
- Or
- When a virtual function is called pure virtual? Write an appropriate program to make your understanding clear.
- c) Write a complete program that shows the uses of virtual function. 4

N.B-> The answer of all these question are available in this lecture sheet

Muhammad Nazim Uddin
 Lecturer (Adjunct)
 Department of CSE, IIUC